

增強課程 6：實作 BAdI (Enhancement Implementation)

Lecture

en05 建好了 Enhancement Spot / BAdI Definition，這題要做的是「實作」——建立真正的 Implementation，讓客製化邏輯真的跑起來。這題準備了兩個案例：自己建的 **ZES_EN05_GREETING** (安全、可控，示範 Multi Use 多實作機制) 與真實標準 BAdI **WORKORDER_UPDATE** (有風險、需要安全閘設計，示範真實企業客製化的完整流程)。

Implementation 建立也是 **GUI-only (SE19)**，跟 en04 的 **Source Code Plugin**、en05 的 **Enhancement Spot** 一樣：直接對 `/sap/bc/adt/enhancements/enhoxh` POST 一路依錯誤訊息補齊 `enho:contentCommon`、`enho:runtimeBehaviorShorttext` 等必要元素後，最後回 `Resource controller does not support method POST`——這個端點本身就不支援 POST，不是資料格式問題，證實建立空殼一定要走 SE19；空殼建好之後的內容讀寫 (含直接指定 Interface 方法邏輯) 可以正常走 ADT。

案例一：ZES_EN05_GREETING 加第二個 Implementation，驗證 Multi Use 執行順序

Multi Use BAdI 沒有 Filter 時，`CALL BADI` 不是「只挑一個 Implementation 執行」，而是依序呼叫所有 **Active** 的 **Implementation**，每一個都拿到前一個處理過的 `CHANGING` 參數繼續往下改——這正是 en05 學到的「Multi Use 不能用 `RETURNING/EXPORTING`」規則存在的原因。本題新增第二個 Implementation **ZIM_EN06_GREETING2** (Class **ZCL_EN06_GREETING2**)，邏輯是把文字追加在 `cv_text` 後面而不是覆蓋。實測結果：

```
Bon voyage on LH! (real implementation ZCL_EN05_GREETING is active) + safe travels,
LH! (2nd implementation ZCL_EN06_GREETING2 also fired)
```

證實兩個 Implementation 依序都執行了 (**ZCL_EN05_GREETING** 先、**ZCL_EN06_GREETING2** 後)，而不是只有一個生效——這是 Multi Use 跟 Single Use 最直觀的行為差異：Single Use 保證「最多一個」，Multi Use 允許「多個同時貢獻結果」。

案例二：真實標準 BAdI **WORKORDER_UPDATE** / **AT_SAVE**

WORKORDER_UPDATE (套件 **COBADI**，真正的 **ENHS/XS**，Multi Use，Interface **IF_EX_WORKORDER_UPDATE**) 是 PM/PP/PS/PI 訂單存檔時的真實掛勾點，方法 **AT_SAVE** 在存檔時被呼叫：

```
methods AT_SAVE
importing
    !IS_HEADER_DIALOG type COBAI_S_HEADER_DIALOG
exceptions
    ERROR_WITH_MESSAGE .
```

查證 **COBAI_S_HEADER_DIALOG** (**COBAI** Type Group 裡的型別) 發現它 **LIKE CAUFVD**——跟 en04 用過的結構完全一樣，代表可以直接沿用 en04 已驗證的安全閘設計：只有 `IS_HEADER_DIALOG-WERKS = '1011'` 且

IS_HEADER_DIALOG-AUART = 'PP71' (教學專用組合) 才動作，其餘一律不做任何事，不影響任何真實 PM/PP/PS/PI 工單存檔。

⚠ 這題端對端驗證過程中踩到一個嚴重到會讓系統當機的真實錯誤——絕對不能在 **BAdI Implementation** 裡下 **COMMIT WORK**：第一版程式碼在寫完稽核記錄後加了一行 **COMMIT WORK**。(想著「保險起見存進去」)，結果使用者一存檔就整個 Dump：

```
Category          ABAP programming error
Runtime Errors     MESSAGE_TYPE_X
ABAP Program       SAPLCOZV
* Unexpected COMMIT WORK!!!
* there should be no COMMIT WORK in order processing before
* fm CO_ZV_ORDER_POST was executed, since this might lead to
* inconsistencies!!!
```

原因：**AT_SAVE** 是在訂單存檔框架 (**SAPLCOZV**) 還沒處理完的交易 (**LUW**) 中間被呼叫的，這個 **LUW** 的擁有者是最上層的存檔框架，不是我們的 Implementation。**COMMIT WORK** 只能由「擁有 **LUW** 的最上層呼叫者」下達；被呼叫的子程式 / Implementation / Function Module 如果自己下 **COMMIT WORK**，等於在別人還沒做完事的時候硬生生把交易切斷，可能讓資料庫留在不一致的中間狀態——**SAP** 標準框架非常清楚這個風險，所以在存檔流程裡主動放了偵測機制，一旦發現不該出現的 **COMMIT WORK** 就直接讓程式當機，寧可讓開發者馬上發現問題，也不讓不一致的資料悄悄進資料庫。拿掉 **COMMIT WORK** 之後，稽核記錄的 **INSERT** 會跟著訂單存檔本身的交易一起被最上層框架 **COMMIT**，不需要 (也不能) 自己額外處理。

修正、重新啟用、真實 **C001** 存檔 (Plant **1011** / Order Type **PP71**) 測試成功，**ZEN06_ATSAVE_LOG** 正確寫入一筆記錄，**AUFNR** 欄位顯示 **%000000000001**——一個附帶的觀察：**AT_SAVE** 觸發當下，工單號碼似乎還是內部暫時性的編號格式 (**%** 開頭)，還沒轉成最終的 12 碼格式，這代表在 **AT_SAVE** 這個時間點，不能假設拿到的工單號碼已經是最終格式，如果客製化邏輯需要用到「正式工單號碼」，可能要考慮換一個更晚的掛勾點 (例如 **IN_UPDATE**，在 Update Task 階段執行，號碼應該已經確定)。

案例三：Filter-dependent BAdI，補齊 **en01/en03** 只講過概念、沒有實機建立過的一塊

en01 / en03 提過 Multi Use BAdI 可以用 **Filter** 依條件 (如公司代碼、工廠) 分流不同 Implementation，但一直停留在概念講解跟情境判斷申論題 (**en03** 題 5：SD 銷售大陸/海外分流定價邏輯)，沒有真的在系統裡建過。本題新建一個獨立的 BAdI (不動 **en05** 既有的 **ZES_EN05_GREETING**，避免影響前面案例)，用航空公司代碼 (**S_CARR_ID**) 當 Filter，讓不同代碼觸發不同 Implementation。

建立流程跟 **en05** 的無 Filter BAdI 大致相同，多一步「Create Filter」：

1. **SE18** 建 Enhancement Spot / BAdI Definition **ZES_EN06_FILTER_DEMO** (Multi Use，Interface 用 ADT 先建好的 **ZIF_EN06_FILTER_GREETING**)
2. 在 **BAdI Definition** 節點右鍵 → **Create Filter** (⚠ 不是勾選 Usability 區塊那個「Limited filter use」核取方塊——那個是另一個用途，用來限制 Filter 值只能是預先列出的固定清單，不是啟用 Filter 本身；Filter 是獨立宣告的物件，靠右鍵選單建立)：
 - **BAdI Filter**：自訂一個名稱 (如 **CARRID**)
 - **Filter Type**：⚠ 這個欄位是單一字元，只能選 SAP 內建的五種基本型別分類之一 (**I** Integer / **C** Character-like / **S** String / **N** Numeric / **P** Packed)，不是填一個完整的 Data Element 名稱 (一開始想填 **S_CARR_ID** 是錯的)——航空公司代碼是 3 碼字元，選 **C** (Character-like)

- **Filter Value Check** : 選 **No Check** 最單純 (要做完整驗證可選 **Automatically by dictionary** , 但要另外掛 Data Element , 本題沒有深入)
3. 存檔、Activate——確認 GET 出來的 XML 有 `<enhs:filter enhs:filterName="CARRID" enhs:filterType="C" .../>`
4. **SE19** 建立 Implementation , 跟 en05 略有不同的操作順序 (⚠ 這是這題排錯發現、之前理解錯的地方) :
- SE19 初始畫面「Create Implementation」區塊 → 選 **New BAdI** → 先填 **Enhancement Spot** 名稱 (**ZES_EN06_FILTER_DEMO**) → 按 Create
 - 彈出「Create Enhancement Implementation」對話框 → 填 **Enhancement Implementation** (容器層級名稱) + Short Text → 確認
 - 再彈出「Create BAdI Implementations for Existing BAdI Definitions」表格 → 這時候才填 **BAdI Implementation** 名稱 + **Implementation Class** (**ZCL_EN06_FILTER_LH**) + 從下拉選單選 **BAdI Definition**
 - 進到 Implementation 的「Filter Values」頁籤 → Create Combination → 填 **Filter=CARRID / Comparator== / Value 1=LH**
 - 雙擊 Implementing Class , 實作 **GET_GREETING** 方法內容 , 存檔 Activate
 - 對 **AA** 重複整個流程 (**ZCL_EN06_FILTER_AA** / Filter 值 **AA**)
 - ⚠ **2026-07-30 補課發現** : 這個 BAdI Implementation 在 TADIR 裡實際登記的物件名稱是「Enhancement Implementation 容器層級的名稱」(第二個對話框填的那個) , 不是第三個表格裡填的 BAdI Implementation 名稱——**AA** 案例當初容器跟 Implementation 剛好取同名 (**ZIM_EN06_FILTER_AA**) , **LH** 案例則是容器叫 **ZEI_EN06_FILTER_LH** , 導致 ADT / TADIR 查得到的是 **ZEI_EN06_FILTER_LH** (透過 `GET /sap/bc/adt/enhancements/enhoxh/<容器名>` 才能讀到 , 裡面 `enho:badiImplementation` 才是實際 Filter 設定) 。兩個對話框的名稱其實是同一件事的兩層包裝 , 建議乾脆取同名 , 避免事後查證要記兩個名字。
5. 測試程式呼叫 `GET BADI go_badi FILTERS carrid = lv_carrid.` ——注意 **FILTERS** 子句用的是 **Filter** 名稱 (**carrid** , 對應 **SE18** 建的 **CARRID**) , 跟方法本身的 **iv_carrid** 參數是兩件事 , 剛好同名容易誤會成同一個東西 , 實際上 Filter 值只是拿來讓框架決定「該呼叫哪個 Implementation」, 不會自動塞進方法參數 , 方法呼叫時該傳的參數還是要自己傳。

實測結果 (**ZR_EN06_FILTER_DEMO**) :

```
CARRID=LH =>
LH 專屬問候語 : Lufthansa greets carrier LH
CARRID=AA =>
AA 專屬問候語 : American Airlines greets carrier AA
CARRID=UA (無 Implementation) =>
UNCHANGED
```

LH / AA 各自正確觸發對應 Implementation ; **UA** 沒有任何 Implementation 掛這個 Filter 值 , **CALL BADI** 安靜地什麼都不做 (**cv_text** 維持呼叫前設定的 **UNCHANGED** 哨兵值) , 這正是 en03 學過的「Multi Use 沒有生效中的 Implementation 也不會報錯」規則 , 在 Filter 情境下的具體體現——差別是這裡的「沒有生效」是「這個 Filter 值沒有對應的 Implementation」, 不是「完全沒人實作」。

學習目標

- 能講出 BAdI Implementation 的建立也是 GUI-only (SE19)，跟 Enhancement Spot、Source Code Plugin 同一個模式：空殼建立要 GUI，內容讀寫可以走 ADT
- 能講出 Multi Use BAdI 沒有 Filter 時的執行行為：所有 Active Implementation 依序執行，每個都能在前一個的 **CHANGING** 結果上繼續處理，不是「只挑一個」
- 能講出「絕對不能在 BAdI Implementation 裡下 **COMMIT WORK**」的原因：LUW 的擁有權屬於最上層呼叫者，子程式自行 COMMIT 會打斷別人未完成的交易，這是會讓系統直接 Dump 的嚴重錯誤，不是隱性 bug
- 能講出如何查證一個 BAdI 方法的參數型別本質（如 **COBAI_S_HEADER_DIALOG LIKE CAUFVD**），並判斷能不能沿用之前課程已驗證過的安全閘設計
- 知道 BAdI 掛勾點被呼叫的「時間點」會影響能拿到的資料完整度（例如 **AT_SAVE** 時工單號碼可能還是暫時格式），設計客製化邏輯前要先確認掛勾點的執行時機
- 能講出 Filter-dependent BAdI 的建立方式：Filter 是在 BAdI Definition 節點右鍵 **Create Filter** 獨立宣告的物件，不是 Usability 區塊的「Limited filter use」核取方塊；Filter Type 只能選 **I/C/S/N/P** 五種基本型別分類，不能填完整的 Data Element 名稱
- 能講出 SE19 建立 Filter-dependent Implementation 的正確操作順序：先在初始畫面填 **Enhancement Spot** 名稱，才輪到填 Implementation 名稱 / Class / 選 BAdI Definition，順序跟直覺（先想 Implementation 叫什麼）容易顛倒
- 能解釋 **GET BADI ... FILTERS <filter名> = <值>** 裡的 Filter 名稱，跟方法呼叫時傳的參數是兩件獨立的事：Filter 只決定「呼叫哪個 Implementation」，不會自動變成方法的參數值

事前準備（已於本系統 client 130 實際完成，非假設）

1. 案例一（承接 en05 物件）：**ZIM_EN06_GREETING2 / ZCL_EN06_GREETING2**（\$TMP，使用者於 SE19 建立骨架，Claude 用 ADT 寫入內容），掛在 en05 的 **ZES_EN05_GREETING** 底下，邏輯是把文字追加到 **cv_text** 後面。已用 **programrun** 無頭執行驗證：**ZR_EN05_FLIGHT_GREETING_DEMO**（加寬 **LINE-SIZE** 避免 Classic List 截斷長字串）顯示兩個 Implementation 依序都執行，**ZCL_EN05_GREETING** 先、**ZCL_EN06_GREETING2** 後。
2. 案例二（真實標準 BAdI）：**ZEN06_ATSAVE_LOG**（自訂稽核表，\$TMP，**WERKS+AUART+AUFNR+LOGDATE+LOGTIME**）、**ZIM_EN06_WORKORDER_ATSAVE / ZCL_EN06_WORKORDER_ATSAVE**（\$TMP，使用者於 SE19 建立骨架，掛在真實標準 Enhancement Spot **WORKORDER_UPDATE** 底下，Interface **IF_EX_WORKORDER_UPDATE**），只在 **WERKS='1011' / AUART='PP71'** 才寫稽核記錄，其餘方法均為空殼、不做任何事。
3. 兩層驗證：
 - 單元測試（**ZR_EN06_ATSAVE_UNIT_TEST**，\$TMP）：不透過真實 BAdI 派送，直接 **CREATE OBJECT** + 呼叫方法，驗證安全閘組合（**1011/PP71**）寫入 1 筆記錄、非安全閘組合（**1011/PP01**）寫入 0 筆記錄，**programrun** 無頭執行驗證成功。
 - 真實存檔測試：使用者用 **CO01**（Plant **1011** / Order Type **PP71**）建立真實工單並存檔，**ZEN06_ATSAVE_LOG** 正確寫入一筆新記錄（**AUFNR=%00000000001**），證實 Enhancement 對真實訂單存檔確實生效。過程中一度因 Implementation 誤含 **COMMIT WORK** 導致真實 Dump（**MESSAGE_TYPE_X**，**SAPLCOZV**），已修正並重新驗證成功。

⚠ 排錯記錄（2026-07-30 補課）：不同真實工單的稽核記錄卻共用同一個 **AUFNR=%00000000001**，是設計缺口不是隨機巧合——使用者在兩個不同日期（7/29、7/30）分別用 **CO01** 建立了兩張不同的真實工單，**ZEN06_ATSAVE_LOG** 卻都記成同一個 **%00000000001**，導致這張稽核表實際上無法用 **AUFNR** 分辨到底是哪一張工單。

- **根本原因**：AT_SAVE 觸發當下，新建工單的 AUFNR 還是一個暫時號碼（內部佔位格式，每次新建工單都重新從這個暫時號碼起算），要等真正的號碼從 Number Range 確定之後，SAP 才會另外呼叫一個專用的掛勾點通知「暫時號碼 → 真實號碼」的對應——查 IF_EX_WORKORDER_UPDATE 完整方法清單，果然找到 NUMBER_SWITCH(I_AUFNR_OLD, I_AUFNR_NEW, I_AUFPL_OLD, I_AUFPL_NEW, I_AUTYP) 這個方法，證實這是 SAP 標準設計好的機制，不是本題獨有的巧合或 bug。
- **修法**：在 NUMBER_SWITCH 裡把 AT_SAVE 當下寫入的暫時號碼記錄，回填成真正的工單號碼：

```
``abap method IF_EX_WORKORDER_UPDATE~NUMBER_SWITCH. IF i_aufnr_old IS NOT INITIAL AND
i_aufnr_new IS NOT INITIAL AND i_aufnr_old <> i_aufnr_new. UPDATE zen06_atsave_log SET
aufnr = i_aufnr_new WHERE aufnr = i_aufnr_old AND werks = '1011' AND auart = 'PP71'.
ENDIF. endmethod.`` 用 werks/auart 縮小範圍，避免動到不相關的既有記錄（Client 由編譯器自動處理，Open SQL 不可在 WHERE 明寫 MANDT，見 .claude/rules/sap-adt-mcp.md` 第 10 節）。
```

- **驗證方式刻意避開真實 CO01**：直接在 ZR_EN06_ATSAVE_UNIT_TEST 加一段測試，模擬「AT_SAVE 寫入暫時號碼 %TESTTEMP01 → NUMBER_SWITCH 回填成 UT_FINAL001」，確認暫時號碼那筆記錄消失、回填後的真實號碼那筆記錄出現，全程不消耗真實 Number Range、不建立真實工單。
- **⚠️ 過程中的操作疏失（誠實記錄）**：驗證新版單元測試程式之前，忘記先把改好的程式碼 sap_set_source 推回 SAP，就直接呼叫 programrun 執行——結果執行到的是舊版本程式（開頭有 DELETE FROM zen06_atsave_log WHERE werks = '1011' AND auart = 'PP71'。這種依 werks/auart 全面刪除的清理邏輯），把 ZEN06_ATSAVE_LOG 裡原本兩筆真實 CO01 測試留下的稽核記錄（7/29、7/30 各一筆）也一併清空了，且已 COMMIT WORK、無法復原。教訓：任何會執行 DELETE / COMMIT WORK 的測試程式，修改後必須先確認新版本已經真的推送並啟用到系統上，才能執行——不能假設本地檔案改了就代表系統上也是新版本；也再次印證「測試程式的清理邏輯應該只鎖定測試專用的特定值（如本例改成的 aufnr = 'UNITTEST001' 等），不要用 werks/auart 這種會波及真實資料的條件做全面 DELETE」。事後已把測試程式的兩處 DELETE 都改成只鎖定測試用 AUFNR 值。
- **案例三（Filter-dependent BAdI）**：ZIF_EN06_FILTER_GREETING（Interface，ADT 建立）+ ZES_EN06_FILTER_DEMO（Enhancement Spot / BAdI Definition，使用者於 SE18 建立，Multi Use，Filter CARRID type C）+ ZE1_EN06_FILTER_LH / ZCL_EN06_FILTER_LH（Filter 值 LH）與 ZIM_EN06_FILTER_AA / ZCL_EN06_FILTER_AA（Filter 值 AA，皆使用者於 SE19 建立，Claude 用 ADT 寫入 Class 內容）。驗證程式 ZR_EN06_FILTER_DEMO 已用 programrun 無頭執行驗證成功：LH / AA 各自觸發對應 Implementation，UA（無 Implementation）維持呼叫前的哨兵值不變。（2026-07-30 補課：LH 的 Implementation 容器物件補建為 ZE1_EN06_FILTER_LH，見上方排錯記錄）

題目需求

1. 解釋 Multi Use 沒有 Filter 時的執行語意：如果有 3 個 Active Implementation 都修改同一個 CHANGING 參數，最終結果會是什麼？這跟「只有一個 Implementation 會生效」的直覺印象有什麼落差？
2. 解釋為什麼 BAdI Implementation 不能下 COMMIT WORK，並說明如果拿掉安全機制（假設 SAP 沒有 SAPLCOZV 那段偵測邏輯），偷偷下 COMMIT WORK 會造成什麼樣的資料風險（提示：想想如果存檔框架後面還有其他表格要更新，你的 COMMIT WORK 提前把交易切斷會發生什麼事）。
3. 解釋為什麼要先做單元測試（直接呼叫 Class 方法），再做真實存檔測試：這個順序具體避免了什麼風險？如果跳過單元測試直接用真實訂單測試安全閘邏輯，有沒有可能造成本題實際發生過的那種當機？
4. AUFNR 在 AT_SAVE 時顯示 %000000000001 這個觀察，對設計 BAdI 客製化邏輯有什麼提醒：如果你的需求是「工單存檔後，把正式工單號碼寫進另一張表」，還適合用 AT_SAVE 這個掛勾點嗎？
5. 解釋 CARRID=UA 那組測試的意義：cv_text 執行前被設成 UNCHANGED，執行後還是 UNCHANGED，這代表了什麼？如果沒有先設這個哨兵值，直接看 cv_text 的內容，能不能同樣確認「沒有任何 Implementation 被呼叫到」？

參考答案

Multi Use 無 Filter 的執行語意：3 個 Active Implementation 會依 SAP 決定的順序（通常是建立順序，但不保證，且沒有官方文件承諾特定順序）依序執行，每一個都收到前一個處理完的 **CHANGING** 參數值繼續處理——最終結果是三個 Implementation 的效果疊加 / 串接，不是「三選一」。這跟直覺常見的「BA_{DI} 只會有一個生效」印象（多半是從 Single Use 或有 Filter 篩選的情境建立的印象）不同：**只要是 Multi Use 且沒有 Filter 區隔，所有 Active Implementation 都會被呼叫**，設計 Implementation 時要考慮到「別人的 Implementation 也可能同時在跑」，不能假設自己是唯一的客製化。

為什麼不能 COMMIT WORK：BA_{DI} Implementation 是被別的程式（存檔框架）呼叫的一段邏輯，執行當下是在框架的 LUW 裡面，框架可能後面還有其他表格更新、其他一致性檢查要做。如果 Implementation 自己下 **COMMIT WORK**，等於把「目前已經做的變更」提前釘死，框架後續如果因為某個原因需要 ROLLBACK（例如後面某個檢查失敗），已經被提前 COMMIT 的部分**沒辦法再撤銷**——資料庫會停在「部分完成、部分沒完成」的不一致狀態（例如工單頭已存但工序資料還沒存，或反過來）。SAP 在 **SAPLCOZV** 裡主動偵測這個風險並讓程式直接 Dump，是刻意設計成「寧可立刻讓開發者發現、也不讓不一致資料默默進資料庫」。

單元測試先行的理由：直接呼叫 Class 方法測試（不透過真實 BA_{DI} 派送）完全不會觸碰到真實訂單存檔的 LUW / 交易框架，即使邏輯有 bug（例如本題原本誤含的 **COMMIT WORK**），最多就是單元測試本身的 **programrun** 執行結果不符預期，**不會影響任何真實資料或觸發框架層級的當機**——因為單元測試呼叫時根本沒有一個「框架的 LUW」存在。本題如果跳過單元測試、直接拿真實訂單測試（其實本題真的是這樣做才踩到 Dump 的，單元測試是*之後*才補上的防護），代價就是使用者的 CO01 交易直接當機，雖然 ABAP Dump 會自動 ROLLBACK 不會留下壞資料，但仍然是一次不必要的、會嚇到使用者的失敗經驗——**先單元測試、確認邏輯正確，再碰真實交易**，是能大幅降低這類風險的順序。

AT_SAVE 時機的提醒：**AUFNR=%000000000001** 這種格式顯示工單號碼在 **AT_SAVE** 當下可能還沒轉成最終格式（% 開頭像是內部暫時性編號的記號）。如果需求是「工單存檔後，把正式工單號碼寫進另一張表」，**AT_SAVE** 可能不是最適合的掛勾點——應該改用 **IN_UPDATE**（在 Update Task 階段執行，通常在這個時間點主要的號碼分配都已經確定）或是先驗證 **AT_SAVE** 拿到的號碼在特定情境下是否已經是最終格式（不同的訂單類型/建立情境可能行為不同，不能一概而論）。這是一個很好的提醒：**BA_{DI} 文件通常只講「什麼時候被呼叫」，不會明講「這個時間點資料完整到什麼程度」，遇到不確定的欄位，實際測試觀察比看文件猜測更可靠**。

UA 哨兵值測試的意義：**cv_text** 在呼叫前被明確設成 **UNCHANGED**，呼叫後如果還是 **UNCHANGED**，代表 **CALL BADI** 這次呼叫完全沒有進入任何 Implementation 的方法本體（沒有任何程式碼有機會修改 **cv_text**）——這是能直接證明「沒有 Implementation 被呼叫」的做法。如果不先設定哨兵值，**cv_text** 只是宣告後的初始空值，執行後如果還是空的，會有歧義：到底是「沒有 Implementation 被呼叫」，還是「有 Implementation 被呼叫，但它自己的邏輯剛好把 **cv_text** 設成空字串」，這兩種情況從結果上無法區分。哨兵值（**sentinel value**）這個測試技巧的重點是：選一個「正常邏輯絕對不會產生」的值當作呼叫前的初始狀態，這樣呼叫後只要看到這個值還在，就能排除「邏輯執行了但恰巧產生同樣結果」的可能性——這跟 en07 學到的「編譯期錯誤不能代替執行期驗證，要看實際輸出」是同一種「用可觀察的證據排除歧義」的驗證思維。

思考題

1. 本題案例一（**ZES_EN05_GREETING**）跟案例二（**WORKORDER_UPDATE**）都是 Multi Use，但案例一我們刻意設計成「兩個 Implementation 都追加文字，順序清楚可見」，案例二則是「只有一個 Implementation，安全閘決定要不要動作」。如果案例二也想做成「多個 Implementation 依序處理」的設計（例如一個負責記錄稽核、另一個負責額外驗證），要注意什麼？（提示：想想如果兩個

Implementation 都要用同一個安全閘條件、但各自獨立開發維護，會不會有條件重複維護、容易漏改其中一處的風險——這是多 Implementation 情境下常見的維護痛點)

2. COMMIT WORK 的教訓具體發生在「訂單存檔框架」這個情境，但這個規則其實適用於**所有**被別人呼叫的程式碼 (Function Module、Method、BAdI Implementation.....)。回想 en02 的 Number Range 教訓 (NUMBER_GET_NEXT 取號非交易性、不受 COMMIT/ROLLBACK 約束)——這兩個「交易正確性」的教訓分別提醒你什麼不同的風險？ (提示：Number Range 的教訓是「即使你 ROLLBACK，某些系統動作仍然生效、不會復原」；這題的教訓是「你自己主動 COMMIT，可能讓別人還沒做完的事被迫提前定型」——兩者都是「交易邊界不是你以為的那樣」，但方向相反)
3. 本題的安全閘設計 (WERKS='1011' 且 AUART='PP71') 跟 en04 用的是同一組測試值。如果之後 en07 (Explicit Enhancement Point) 或期末綜合實作也需要用到「真實但安全」的測試資料，你會建議繼續沿用這組 1011/PP71，還是每題各自另外找一組？為什麼？ (提示：沿用同一組的好處是「已知安全、不用每次重新確認」；缺點是如果之後不同题目的 Enhancement 同時掛在同一個廠別/製令類型組合上，彼此可能互相干擾、難以個別驗證——需要視題目之間會不會同時生效來決定)
4. 案例三的安全閘設計跟案例二不一樣：案例二是**程式碼裡寫 IF 判斷** (WERKS='1011' AND AUART='PP71' 才動作)，案例三是用 **Filter 讓框架自動決定該呼叫哪個 Implementation**，兩者都能達到「依條件分流」的效果。如果案例二也改用 Filter (例如用「工廠代碼」當 Filter，不同工廠掛不同 Implementation)，會遇到什麼案例三沒有的限制？ (提示：案例二的核心邏輯是「只有一組條件動作，其餘都不動作」，這其實比較接近 Single Use / 有安全閘的 Multi Use；案例三是「每個 Filter 值各自對應完全不同的邏輯內容」，如果案例二真的要拆成多個 Filter 分流的 Implementation，代表每個工廠都要各自維護一份完整的稽核邏輯，程式碼會重複，除非把共用邏輯抽成一個共用的 Class 方法讓每個 Implementation 呼叫)